

Foundations of Computing Systems Laboratory

Laboratory Test 1

Date: 3 September 2026**Time:** 2:30 p.m.–4:30 p.m.**Duration:** 2 hours**Name:** _____**Roll number:** _____**Instructions**

- All three questions are compulsory.
- Write the requested HDL or assembly-language files and test them using the supplied tools and test scripts.
- Use bit 0 as the least significant bit in all 16-bit chips.
- Unless stated otherwise, chips completed in an earlier part may be used in a later part.

Question 1: Subtractor Chips [5+5+5 marks]**(a) Half-Subtractor**

A half-subtractor subtracts a one-bit input b from a one-bit input a . It has two one-bit outputs: **diff**, the difference bit, and **borrow**, the borrow bit.

a	b	diff	borrow
0	0	0	0
0	1	1	1
1	0	1	0
1	1	0	0

Implement the chip **HalfSubtractor** using only And, Or, and Not gates. Your chip must have the following interface:

```
CHIP HalfSubtractor {
    IN a, b;
    OUT borrow, diff;
}
```

(b) Full-Subtractor

A full-subtractor subtracts the one-bit input b and the incoming borrow c from the one-bit input a . It produces a difference bit **diff** and an outgoing borrow bit **borrow**.

a	b	c	diff	borrow
0	0	0	0	0
0	0	1	1	1
0	1	0	1	1
0	1	1	0	1
1	0	0	1	0
1	0	1	0	0
1	1	0	0	0
1	1	1	1	1

Implement the chip **FullSubtractor**. You may use the **HalfSubtractor** chip developed in part (a) and the available basic gates. Your chip must have the following interface:

```
CHIP FullSubtractor {
    IN a, b, c;
    OUT borrow, diff;
}
```

(c) **16-Bit Subtractor**

Implement the chip **Sub16**, which computes the 16-bit subtraction $a - b$. You may use the **HalfSubtractor** and **FullSubtractor** chips developed in part (a), (b) and the available basic gates.

```
CHIP Sub16 {
    IN a[16], b[16];
    OUT diff[16];
}
```

The bus `diff[16]` must contain the 16-bit result.

Question 2: A 16-Bit Register with Left Shift [15 marks]

Implement a 16-bit register that can load a new value, *shift its stored value one position to the left*, or retain its current value. In a left shift, every bit moves one position towards the most significant end and the rightmost bit becomes 0.

Chip name: ShiftRegister16

Inputs: `in[16]`, `load`, `leftShift`

Outputs: `out[16]`

Function:

$$\text{out}(t) = \begin{cases} \text{in}(t-1), & \text{if } \text{load}(t-1) = 1, \\ \text{LS}(\text{out}(t-1)), & \text{if } \text{load}(t-1) = 0 \text{ and } \text{leftShift}(t-1) = 1, \\ \text{out}(t-1), & \text{otherwise.} \end{cases}$$

On a 16-bit number x , the operation $\text{LS}(x)$ returns the number obtained by shifting x one position to the left and filling the rightmost bit with 0. For example, $\text{LS}(1011\ 0010\ 1101\ 0110) = 0110\ 0101\ 1010\ 1100$.

Formally, if $y = \text{LS}(x)$, then

$$y[0] = 0, \quad y[j] = x[j - 1] \quad \text{for every } j > 0.$$

Thus, `load` has priority over `leftShift`: if both inputs are 1, the chip loads `in[16]`.

Use the following interface:

```
CHIP ShiftRegister16 {  
    IN in[16], load, leftShift;  
    OUT out[16];  
}
```

Test all three operations: load, left shift, and retain.

Question 3: Sum of the First n Natural Numbers [20 marks]

Write a Hack assembly-language program that computes

$$1 + 2 + \cdots + n.$$

The input n is stored in `RAM[1]`. Your program must read this value, compute the sum, and store the result in `RAM[2]`. If $n = 0$, the result must be 0. You may assume that $n \geq 0$ and that the result fits in a 16-bit signed integer. After storing the result, the program must enter an infinite loop.

Save your program as `sum.asm` and proceed as follows:

1. Write the assembly program in the editor.
2. Use the assembler to translate the program. If syntax errors are reported, correct the source file and assemble it again.
3. Use the assembler's **Load in CPU Emulator** button to open the assembled program in the emulator.
4. In the CPU emulator, choose **Load a test script** and select the supplied `sum.tst` file.
5. Run the test. If the result is wrong or a run-time error occurs, edit the source, assemble it again, and rerun the emulator test.

Some useful chips

Command	Meaning
And(a=x, b=y, out=z);	Compute the bitwise AND of x and y and store the result in z .
Or(a=x, b=y, out=z);	Compute the bitwise OR of x and y and store the result in z .
Not(in=x, out=y);	Compute the bitwise NOT of x and store the result in y .
Bit(in= x, load= y, out= z);	If $y = 1$, store the value of x in z . Otherwise, store the current value in its memory in z .
Register(in= x, load= y, out= z);	If $y = 1$, store the value of x in z . Otherwise, store the current value in its memory in z . Here, x is a 16-bit input.

Some useful Hack assembly commands

Command	Meaning
@253	Load the constant 253 into the A register.
@xyz	Load into A the number or address represented by the symbol xyz . The symbol may denote a variable or a label.
D=M	Load the value stored in RAM[A] into D.
M=-1	Store the value -1 in RAM[A] .
M=M+1	Add 1 to the value stored in RAM[A] .
D=D+M	Add the value stored in RAM[A] to D and store the result in D.
M=D+M	Add the value stored in D to RAM[A] and store the result in RAM[A] .
D=D-A	Subtract the value stored in A from D and store the result in D.
D=D-1	Subtract 1 from D and store the result in D.
D=D&A	Perform a bitwise AND of D and A and store the result in D.
D=D A	Perform a bitwise OR of D and A and store the result in D.
D; JGT	If D is greater than 0, jump to the ROM address currently stored in A.
D; JLT	If D is less than 0, jump to the ROM address currently stored in A.
0; JMP	Jump unconditionally to the ROM address currently stored in A.

End of question paper